1	2	3	$\sum$

Freie Universität Berlin

Algorithmen, Datenstrukturen und Datenabstraktion (WiSe 18/19)

Docent: Prof. Dr. Wolfgang Mulzer

Tutorin: Valeria Zahoransky; Tutorial: 07

Exercise 04

Release: 02.11.2018; Due: 17.11.2018

Viktoriya Kraleva, Samuel Enderwitz

---

1 Java-Repetition (__/15)

(a) *Name two Java keywords each that are associated with the object-oriented concepts* encapsulation *and* inheritance*, respectively.*

- Kapselung
  1. private
  2. protected
- Vererbung
  1. extends
  2. super

(b) *Name three kinds of* polymorphism *that are implemented in Java and provide a short example for each.*

1. Überladene Funktionen

   Die Funktion + ist für verschiedene Datentypen definiert(Int, Long, Flot, Double)

2. Vererbungspolymorphie

   Eine Klasse hat eine Funktion definiert. Eine Unterklasse, die diese Klasse extentiert kann diese Funktion überschreiben.

3. Parametrische Polymorphie

   Wenn ein Objekt aus einer Klasse gebildet wird und Werte bei der Initialisierung übergeben wird. Dieses Konzept wird auch Generizität genannt.

(c) *Describe in two sentences what we mean by* annotations *in Java. What is the* `@Override` *annotation? Why would you use it?*

Annotationen erlauben die Einbindung von Metadaten im Programmcode. Der Compiler kann dann anhand von diesen Metadaten weiterer Code oder Dateien erstellen oder warnings und exceptions unterdrücken oder auslösen.

Mit der `@Override` Annotation kann man kennzeichnen, dann man mit dieser Funktion eine Funktion in der Oberklasse überschreiben. Der Compiler gibt einen Fehler auf, wenn eine solche Methode nicht in der Oberklasse vorhanden ist.

(d) *What is the difference between the dynamic and the static type of a (reference)-variable in Java.*

Bei einem **Dynamischen Typen** ist der Typ der Variable erst zur Laufzeit des Programms bekannt. Beim **statischen Typ** dagegen schon bei der Variablendeklaration also bei der Complilierung.

(e) *Consider the following interface and implementing classes.*

```
interface I {
  void a();
}
class A implements I {
  public void a() { System.out.println("A"); }
}
class B implements I {
  public void a() { System.out.println("B"); }
  public void b() { System.out.println("C"); }
}
```

For each line in the following `main`-method, determine whether it is valid or not. Remove the invalid lines, and provide the output of the remaining program. Moreover, for each variable, give its static and (possibly changing) dynamic type. Try to solve this exercise without the help of a computer.

Aus einer statischen Umbegung kann man keine aus nicht statischen Klassen Objekte erzeugen. Somit müssten wir jede Zeile löschen. Deshalb gehen wir davon aus, dass unsere Klassen A und B statisch sind oder unsere Methode nicht statisch.

```
public static void main(String[] args) {
  A a = new A();  // erlaubt
  a.a();  // erlaubt
  a.b();  // nicht erlaubt; b() nicht für A definiert
  B b = new B();  // erlaubt
  b.a();  // erlaubt
  b.b();  // erlaubt
  I i;  // erlaubt
  i = new I();  // nicht erlaubt; Objekt nicht aus Interface erzeugbar
  // ein Interface ist eine Abstrakte Klasse
  i = a;  // erlaubt
  i.a();  // erlaubt
  i.b();  // nicht erlaubt, da b() nicht für I definiert ist
  i = b;  // erlaubt
  i.a();  // erlaubt
  i.b();  // nicht erlaubt, da b() nicht für I definiert ist
  b = i;  // nicht erlaubt; I ist keine Unterklasse von B
  b = (B)i;  // erlaubt i ist schon B
  a = (A)i;  // nicht erlaubt; B ist keine Unterklasse von A
}
```

Die Main-Funktion ohne Fehlerhafte Zeilen.

```
public static void main(String[] args) {
  A a = new A();
```

```
  a.a();
  B b = new B();
  b.a();
  b.b();
  I i;
  i = a;
  i.a();
  i = b;
  i.a();
  b = (B)i;
}
```

Meine Ausgabe ist: (Neue Zeile anstatt Leerzeichen, da println)

A B C A B

(f) *Repeat the task from (e) with the following code fragment.*

```
public static void main(String[] args) {
  List<?> x = new ArrayList<String>();
  List<Object> y = new ArrayList<Integer>();
  List<? extends Number> y2 = new ArrayList<Integer>();
  Object[] z = new Integer[3];
  z[2] = "ALP3";
}
```

2 O-Notation (__/5)

For each of the following statements, decide whether it is true or false. Justify your answer with a proof or a counterexample.

(a) *We have* $\log n \in O(2^{\log \log^2 n})$
Hier werden wir dies mit einem Gegenbeispiel beweisen unsere Vermutung läutet, dass ab n= 10 die Ungleichung nicht stimmt und aus dem Grund,dass in der Aufgabenstellung es gesagt ist, dass man dies mit gegenbeispiel beweisen kann, machen wir es.
$2^{\log(\log^2(100))} = 1,5178...$
$\log_1 0100 = 2$
Damit haben wir dies mit Gegenbeispiel bewiesen. Noch 1 :
$2^{\log(\log^2(100))} = 1,9375...$
$\log_1 01000 = 3$ Damit ist die Aussage false. Der Beweis dafür ist sehr einfach. Wir müssen beweisen, dass ab $n > n_0 \wedge c > 0$ $f(n) > c * g(n)$
Wir ersetzen $f(n)$ mit $log(n)$ und $g(n)$ mit $2^{\log(\log^2(n))}$
Damit erhalten wir :
$\log(n) > 2^{\log(\log^2(n))}$
(Wir lassen c weg, damit wir dies danach bestimmen können.)
Wir logarithmieren die beiden Seiten der Ungleichung mit log_2 (weil wir wissen,dass n ganzzahlig ist, wissen wir auch,dass $\log(n) > 0$ und wir verändern die Ungleichung nicht) und damit erhalten wir:
$\log_2 \log(n) > \log(\log^2(n))$

wir ersetzen log(n) mit x und damit die Gleichung übersichtlicher sein kann.
$\log_2 x > \log(x^2)$ Wir benutzen hier der Basiswechsel Regel für Logarithmus.
$\frac{\log x}{\log 2} > 2 * \log x$
$\log x > (2 * \log 2) * \log x$
Damit ist offensichtlich die Gleichung erfüllt,weil $2 * \log 2$ genau unser c wäre und $c = 2 * \log 2 \approx 0,60$

(b) *We have $f(n) \in O(g(n))$ if and only if $g(n) \in \Omega(f(n))$.*$f(n) \in O(g(n))$ heißt, dass $\exists c > 0 : \forall n > n_0 f(n) \leq c * g(n)$
Damit wenn wir die Definition der Omeganotation betrachten :
$f(n) \in \Omega(g(n))$ heißt, dass $\exists c_0 > 0 : \forall n > n_0, f(n) \geq c * g(n)$
Wenn wir in dem zweiten Formell $f(n)$ mit $g(n)$ ersetzen,bzw $g(n)$ mit $f(n)$ muss :
$\Leftrightarrow \exists c_0 > 0 : \forall n > n_0 g(n) \geq c_0 * f(n)$
Aus dem Grund,dass wir wissen, dass c_0 positiv ist, deswegen dürfen wir die Ungleichug dadurch dividieren.
$\Leftrightarrow \exists c_0 > 0 : \forall n > n_0, \frac{g(n)}{c_0} \geq f(n)$
Wir wissen,dass $g(n) * c \geq f(n)$,deswegen wählen wir $c_0 = \frac{1}{c}$ und damit erhalten wir :
$\Leftrightarrow \exists c > 0 : \forall n > n_0, f(n) \leq c * g(n)$ also die Erste Gleichung. Damit kann mann als Schlussfolgerung: $f(n) \in O(g(n)) \Leftrightarrow \exists c > 0 : \forall n > n_0, f(n) \leq c * g(n) \Leftrightarrow g(n) \in \Omega(f(n)) \Leftrightarrow \exists c_0 = \frac{1}{c} > 0 : \forall n > n_0, g(n) \geq \frac{1}{c} * f(n)$ und damit $f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$
Damit ist die Aussage wahr.

(c) *If $f(n) \in O(g(n))$, then also $2^{f(n)} \in O(2^{g(n)})$.*
Wenn $f(n) \in O(g(n))$, dann muss :
$\exists c > 0 : n > n_0 : f(n) \leq c * g(n)$
Damit erläutert, können wir die zweite Gleichung betrachten :
$2^{f(n)} \in O(2^{g(n)}$
Damit diese Aussage erfüllt ist muss :
$\exists c : n > n_0 : 2^{f(n)} \leq c_2 2^{c*g(n)}$
Wir nehmen nur die Ungleichung von dieser Aussage und teilen durch $2^{c*g(n)}$ (Wir dürfen dies machen, weil wir vom vorher wissen,dass $g(n) > 0$ und $c > 0$
$\exists c : n > n_0 : \frac{2^{f(n)}}{2^{g(n)}} \leq c_2$
Wir kalkulieren den Grenzwert davon :
$\lim_{x \to \infty} \frac{2^{f(n)}}{2^{c*g(n)}} = 0 \leq c_2$
Wir wissen,dass der Grenzwert 0 ist, weil wir vom $f(n) \in O(g(n))$ wissen ,dass $f(n) \leq c * g(n)$
Damit muss auch $2^{f(n)} \leq 2^{c*g(n)}$ (mann kann dies beweisen, indem man die Ungleichung mit $\log_2$ logarithmiert. Damit ist unser $c_2 = 1$ und die Bedingung wird immer erfüllt.
Damit haben wir bewiesen,dass $f(n) \in O(g(n))$.

3 Amortized Analysis (__/10)

Suppose we have a binary counter with an arbitrarily large number of digits from $\{0, 1\}$, all initially 0. To switch one digit, we require one unit of power. How many units of power do

we need in total to count from 0 *to* n *in increments of one?*

(a) *Use the accounting method to show that the total power cost is* $O(n)$.

(b) *Compute the total power cost exactly, assuming that* n *is a power of two.*Wir werden probieren, das Muster zu erkennen und dann zu beweisen, dass dieses Muster immer Funktioniert. Wir betrachten einige Werten.
$T(2^0) = T(1) = 1$ Stromeinheiten
$T(2^1) = T(2) = 2$ Stromeinheiten
$T(2^2) = T(4) = 3$ Stromeinheiten
$T(2^4) = T(2) = 4$ Stromeinheiten
Also $T(n) = T(n-1) - 1$
Aber wir vermuten, dass die nicht-rekursive Formel so aussehen muss, aus dem Grund,dass wir die Voraussetzung haben, dass n eine Potenz von 2 ist:
$T(n) = \log_2 n + 1$
Wenn man die Werten von Oben einsetzt, kann man sicherlich sagen, dass die Aussage wahr ist ,trotzdem werden wir dies mit Hilfe von vollständiger Induktion beweisen:
I.A. n = 1
$T(1) = \log_2 1 + 1 = 0 + 1 = 1$ - also richtig
I.V. Wahr für $n = 2^k$
I.S. $n = 2^k \Rightarrow n_1 = 2^{k+1}$
$T(n_1) = T(n) + 1 = \log_2 n + 1 + 1$
$= \log_2 2^k + \log_2 2 + 1 =$
$\log_2 2 \cdot 2^k + 1 = \log_2 2^{k+1} + 1 =$
$\log_2 n_1 + 1$ - wahr
Und dies ist das genaue Anzahl an Stromeinheiten,die gebraucht sind.